

Dossier de conception

S.I. Gestion d'un Hub évènementiel — Lot 1 (v1)

Révision	Rédacteur	Date	Objet
1	Quentin Hamonic	28/08/2026	Création du document

Introduction

Objet du document

Ce document présente les questions techniques liées à la mise en œuvre de la plateforme de gestion d'un hub évènementiel pour une fédération associative régionale.

Il présente l'architecture logicielle retenue, le diagramme de classes métier, le modèle physique de données et les règles de développement applicables.

Architecture logicielle

Langages, frameworks et librairies spécifiques

Nom	Version	Usage
Java	21 (LTS)	Langage back
Spring Boot	4.0	Socle applicatif, injection de dépendances
Spring Data JPA / Hibernate	4.0 / 7.0	Persistance
Spring Security	7.0	Authentification, autorisation par rôle, hachage Argon2id
springdoc-openapi	2.6	Documentation de l'API (Swagger UI)
JUnit	5.10	Tests unitaires
Maven	3.9	Build, profils dev et prod
Spring Data MongoDB	4.3	Accès à la vue de lecture
Angular	17	Front web responsive

L'usage de Lombok est interdit sur ce projet.

Serveur de base de données

Nom	Version	Rôle
MySQL	8.4 LTS	Source de vérité, règles métier et transactions
MongoDB	7.0	Vue de lecture, périmètre à définir

Focus technique

Diagramme de classes métier

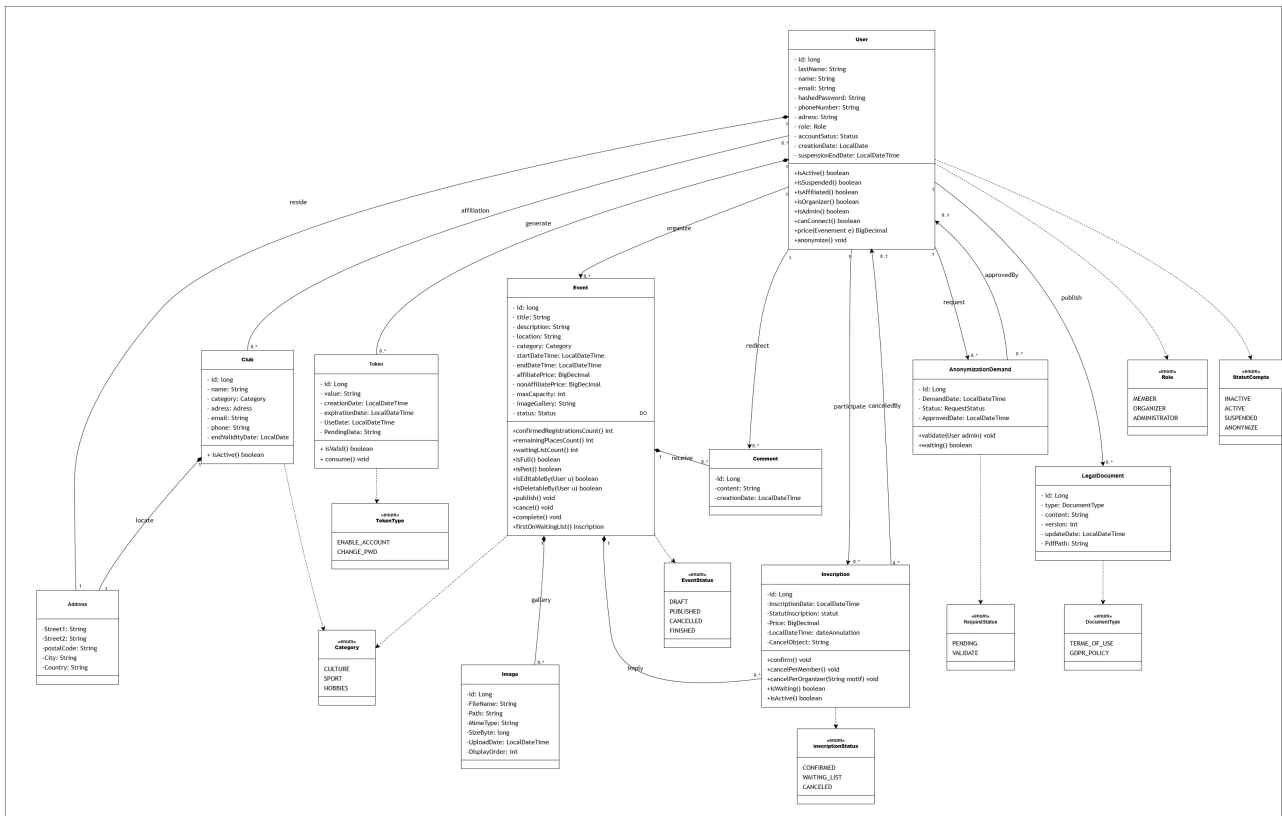


Figure 1 — Diagramme de classes métier

Le modèle est entièrement plat, sans aucun héritage. Les profils (membre, organisateur, administrateur) sont portés par l'attribut `role` de la classe `User`, et non par des sous-classes.

Inscription est une classe d'association entre User et Event. Elle porte la date d'inscription, qui donne l'ordre de la liste d'attente, ainsi que le statut et le tarif figé au moment où la personne s'inscrit.

Modèle physique de données

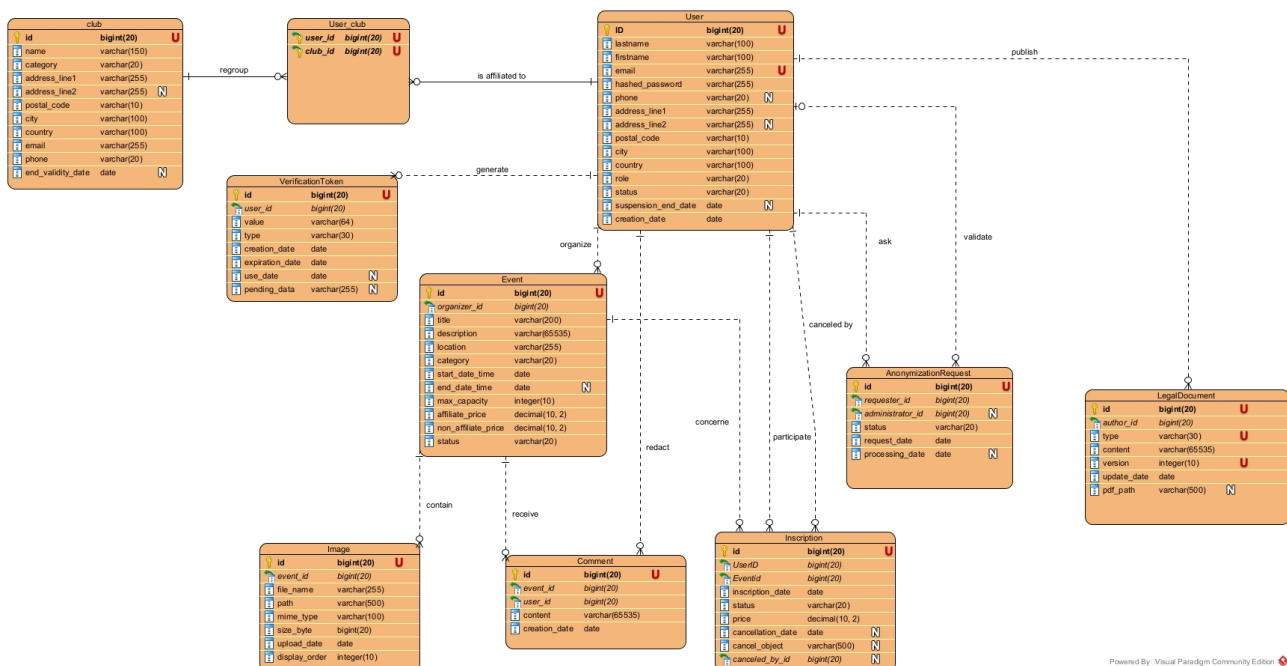


Figure 2 — Modèle physique de données

Table	Contenu
utilisateur	Comptes, tous profils confondus. Adresse fusionnée en colonnes.
club	Clubs affiliés à la fédération.
utilisateur_club	Table de liaison des affiliations. Clé primaire composite.
token_verification	Jetons à usage unique pour l'activation d'un compte et le changement de mot de passe.
evenement	Évènements, avec tarifs et statut du cycle de vie.
image	Galerie d'un évènement.
commentaire	Commentaires déposés sur un évènement.
inscription	Inscriptions et liste d'attente. Traduction de la classe d'association.
demande_anonymisation	Demandes RGPD et leur validation.
document_legal	Conditions d'utilisation et politique RGPD, versionnées.

Les énumérations ne deviennent pas des tables. Elles sont traduites en colonnes VARCHAR contraintes par un CHECK, et mappées en Java avec EnumType.STRING.

Mise en place de la base de données

La table utilisateur est donnée ici en exemple. Le script complet suit les mêmes conventions pour les neuf autres tables.

```
CREATE TABLE utilisateur (
  id          BIGINT      NOT NULL AUTO_INCREMENT,
  nom         VARCHAR(100) NOT NULL,
  prenom      VARCHAR(100) NOT NULL,
  email       VARCHAR(255) NOT NULL,
  mot_de_passe_hache VARCHAR(255) NOT NULL,
  telephone   VARCHAR(20)  NULL,
  adresse_ligne1 VARCHAR(255) NOT NULL,
  adresse_ligne2 VARCHAR(255) NULL,
  adresse_code_postal VARCHAR(10) NOT NULL,
  adresse_ville VARCHAR(100) NOT NULL,
  adresse_pays VARCHAR(100) NOT NULL DEFAULT 'France',
  `role`      VARCHAR(20)  NOT NULL,
  statut      VARCHAR(20)  NOT NULL DEFAULT 'INACTIF',
  date_fin_suspension DATE    NULL,
  date_creation DATETIME   NOT NULL DEFAULT CURRENT_TIMESTAMP,

  PRIMARY KEY (id),
  UNIQUE KEY uk_utilisateur_email (email),
  CONSTRAINT ck_utilisateur_role
    CHECK (`role` IN ('MEMBRE', 'ORGANISATEUR', 'ADMINISTRATEUR')),
  CONSTRAINT ck_utilisateur_statut
    CHECK (statut IN ('INACTIF', 'ACTIF', 'SUSPENDU', 'ANONYMISE'))
) ENGINE=InnoDB;
```

Brique NoSQL, catalogue en vue de lecture

La brique NoSQL sera MongoDB. Son périmètre exact reste à définir et fera l'objet d'une décision ultérieure.

Règles de développement côté back

- L'application est découpée en quatre couches nommées controller, service, repository et entity. Aucune requête en base n'est écrite depuis un controller.
- Les entités ne sortent jamais de la couche service, et les controllers échangent uniquement des DTO.
- Les règles métier qui ne dépendent que d'un objet vivent dans l'entité (estComplet, estPasse, estModifiablePar).
- Les règles qui coordonnent plusieurs entités vivent dans le service (promotion depuis la liste d'attente, envoi des mails, anonymisation).
- Lombok étant interdit, les getters, setters, constructeurs, equals et hashCode sont écrits à la main ou générés par l'IDE.

- equals et hashCode basés sur l'identifiant uniquement, avec traitement explicite du cas id null.
- Constructeur vide protected sur chaque entité, requis par JPA.
- FetchType.LAZY par défaut sur toutes les associations.
- Indentation 4 espaces, une classe par fichier, nommage en anglais côté code.

Tests et intégration

Stratégie de tests

Les tests unitaires sont écrits avec JUnit 5 et portent en priorité sur les classes qui vérifient les règles de gestion, sans base de données ni contexte Spring.

- Sur Event, les méthodes estComplet, estPasse, estModifiablePar et nbPlacesRestantes.
- Sur User, les méthodes estActif, estSuspendu, estAffilie et tarifApplicable.
- Sur Inscription, les transitions de statut et les conditions d'annulation.
- Sur Token, la validité et l'expiration du jeton.

La promotion depuis la liste d'attente et le calcul des places disponibles font l'objet de tests d'intégration avec base de données, car ils dépendent de la concurrence.

Indicateurs de qualité de code

À compléter dans une phase ultérieure du projet.

Règles de sécurité

- Mots de passe hachés avec Argon2id, via l'implémentation native de Spring Security 7. Aucune dépendance à Bouncy Castle.
- La colonne mot_de_passe_hache est dimensionnée en VARCHAR(255), la longueur d'un hachage Argon2id encodé dépassant celle d'un hachage bcrypt.
- Le hachage ne figure jamais dans un DTO de sortie.
- Un mot de passe compte au minimum 12 caractères, dont une majuscule, un chiffre et un caractère spécial.
- Jetons de vérification générés avec SecureRandom, en s'assurant que l'algorithme sous-jacent est NativePRNG et non un générateur logiciel de repli.
- Chaque jeton est à usage unique et porte une date d'expiration.
- Message d'erreur de connexion identique que l'adresse existe ou non.
- Les routes publiques couvrent la consultation des événements, les CGU, la politique RGPD, la connexion et la création de compte.
- Toutes les autres routes exigent une authentification. Les routes d'administration exigent le rôle ADMINISTRATEUR, la gestion d'évènement le rôle ORGANISATEUR.
- Vérification côté serveur de la propriété d'un événement avant toute modification ou suppression.